

Reconsider parallel ranges `::rotate_copy` and `::reverse_copy`

Document #: P3709R0 [[Latest](#)] [[Status](#)]
Date: 2025-05-18
Project: Programming Language C++
Audience: LEWG
Reply-to: Ruslan Arutyunyan
<ruslan.arutyunyan@intel.com>
Alexey Kukanov
<alexey.kukanov@intel.com>

Contents

- [1 Introduction](#)
- [2 `rotate\_copy` recommended design](#)
- [3 `reverse\_copy` recommended design](#)
- [4 Alternative design](#)
- [5 Acknowledgements](#)
- [6 References](#)

§ Abstract

This paper proposes reconsidering the design for parallel ranges `::rotate_copy` and ranges `::reverse_copy` in [\[P3179R7\]](#).

§ 1 Introduction

LEWG approved “range-as-the-output” design aspect for parallel range algorithms proposal [\[P3179R7\]](#). A common questions for such algorithms is what to return for the input when the output size is not sufficient? For the simple cases the answer is pretty simple: we return past the last iterator for the input and it points to the position right after the last copied element.

Consider `std::ranges::copy` as an example:

```
std::vector input{1,2,3,4,5};
std::vector<int> output(5); // output with sufficient size
std::vector<int> smaller_output(3); // output with insufficient size

auto res1 = std::ranges::copy(std::execution::par, input, output);

// after copy invocation res.in == to input.end() is true

auto res1 = std::ranges::copy(std::execution::par, input, smaller_output);

// after copy invocation res.in == to input.begin() + 3 is true
// res.in point to element equal 4
```

Having that result gives us consistent behavior with serial range algorithms, when the output size is sufficient and *stop point* in the input when the output size is insufficient.

The proposed design in [\[P3179R7\]](#) follows the same logic (returning past the last iterator for the input) for parallel ranges `::rotate_copy` and ranges `::reverse_copy` and both algorithms use the same return type as their serial range counterparts. It’s consistent with the rest of the parallel range algorithms that have an output. However, it leads to interesting consequences for the following reasons:

- `reverse_copy` goes in reverse, so past the last iterator for input is never last (unless the output is empty).
- `rotate_copy` has two “subranges” within one range split by *middle*. The past the last iterator for input is never last again: it’s either one of the iterators in `[middle, last - 1]` or in `[first, middle]`.

The proposed design in [\[P3179R7\]](#) looks correct in isolation because users want to know where the algorithm stops if the output size is not sufficient to copy all the elements from the input. The potential problem, however, comes from the combinations of three factors:

- serial range algorithms always return `last` for both `reverse_copy` and `rotate_copy`.
- the return type for serial and parallel range algorithms is the same, however parallel range algorithms return a different iterator for the input even if the output size is sufficient
- we envision serial range algorithms with “range-as-the-output” design in the future, where it is applicable to return both *stop point* and *last*, if it was calculated.

To elaborate more on serial range algorithms: the current strategy for them (for the vast majority) is returning as much information is possible. For example: `ranges::for_each` returns past the last iterator because that is what the algorithm calculated besides applying a callable object element-wise. This is useful information because users might not have such an iterator before the algorithm call; they might only have `sentinel`.

Furthermore, we expect serial range algorithms with “range-as-the-output” design in the future that supports more than `random_access_iterator` only. For such algorithms it totally makes sense to return both the input *stop point* and the input *last*, if the latter was calculated. For the vast majority of algorithms with the output the input *last* is either cannot be calculated (when the output size is insufficient) or it matches the end of input. For `reverse_copy` and `rotate_copy` this is not true. We need to keep that in mind when designing these algorithms, because it would be very unfortunate if “rangified” serial range algorithms end up returning something different from parallel range algorithms. Thus, parallel range algorithms design needs to accept the fact that they should to return more information for `random_access_iterator|range`.

As the reminder, one of the design goals for [P3179R7] was to keep the same return type for the proposed algorithms compared to existing range algorithms, where possible. However, there is a clear indication that we need to do something different for `reverse_copy` and `rotate_copy`.

§ 2 rotate_copy recommended design

In [P3179R7] `rotate_copy` returns past the last inserted copied element for input. By design, it never returns *last*; it returns *middle* for two scenarios:

- output size is sufficient to copy everything from input.
- output size is empty.

The raised concern is that the users might be surprised at runtime by completely different return value when just passing an execution policy function argument. Consider the following example:

Serial vs Parallel `rotate_copy`

iterator-as-the-output <code>rotate_copy</code>	range-as-the-output <code>rotate_copy</code>
<pre>std::list v{1,2,3,4,5,6}; std::vector<int> out_v(6); // view is not common_range auto view = std::views::counted(v.begin(), 6); static_assert(!std::ranges::common_range<decltype(view)>); auto it = view.begin(); std::ranges::advance(it, 3); // iterator as the output auto res = std::ranges::rotate_copy(view, it, out_v.begin()); // res.in is last auto val = std::reduce(view.begin(), res.in); // val is 21</pre>	<pre>std::list v{1,2,3,4,5,6}; std::vector<int> out_v(6); // view is not common_range auto view = std::views::counted(v.begin(), 6); static_assert(!std::ranges::common_range<decltype(view)>); auto it = view.begin(); std::ranges::advance(it, 3); // range as the output auto res = std::ranges::rotate_copy(view, it, out_v); // res.in is middle auto val = std::reduce(view.begin(), res.in); // val is 6</pre>

Please note, that in the table above we don’t use algorithm overloads with execution policy. Instead, we show the future serial range algorithms with “range-as-the-output”. The reasons for that are:

- we need a non-`common_range`, non-`random_access_range`, so that calculating *last* would make sense.
- parallel range algorithms require `random_access_range`, so the code would fail to compile.

While we could say that for `rotate_copy` we would jump to the end to return *last* when the output size is sufficient there two problems with that approach:

- it is inconsistent with other algorithms with “range-as-the-output”. The rest returns past the last iterator.
- for the envisioned serial `ranges::rotate_copy` with “range-as-the-output” we calculate `last` as the iterator but don’t return that. See [Introduction](#) for more information.

So, if we don’t want to surprise users with the different behavior of parallel `rotate_copy` at runtime, we need to make it fail to compile when the returned value for existing `rotate_copy` was stored and used. We expect that users to likely store the return value from range algorithms as `auto` type, so the names of the publicly accessible fields should also be different for parallel range algorithms, compared to the serial ones.

When we introduce “range-as-the-output” with its own bound we take into account that the output size may be less than input size. Since `rotate_copy` has two “subranges” - from middle to last and from first to middle - we want to say where the algorithm stops for both of them, thus we propose to change the result type to “alias-ed” `ranges::in_in_out_result`. We could introduce some new structure because currently `in_in_out_result` is used for the algorithms with two input ranges only. However, we don’t see a reason to introduce yet another type since we think that `in_in_out_result` has a perfect applicability for the discovered use case.

The signatures are below:

```
template <class In, class Out>
using rotate_copy_truncated_result = std::ranges::in_in_out_result<In, In, Out>;

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        random_access_iterator O, sized_sentinel_for<O> OutS>
requires indirectly_copyable<I, O>
ranges::rotate_copy_truncated_result<I, O>
    ranges::rotate_copy(Ep&& exec, I first, I middle, S last, O result, OutS result_last);

template<execution-policy Ep, sized-random-access-range R, sized-random-access-range OutR>
requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>>
ranges::rotate_copy_truncated_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
    ranges::rotate_copy(Ep&& exec, R&& r, iterator_t<R> middle, OutR&& result_r);
```

The design above addresses all the concerns in our opinion because:

- for `in_out_result` the name for input is `in` data member, while for `in_in_out_result` the names for input are `in1` and `in2` data members, so we achieve the goal for the the code to fail to compile when one uses the output and switches to the parallel overload even if users store the result as `auto` type (see a caveat below).
- it takes into account future serial range algorithms with “range-as-the-output” where `rotate_copy` returns
 - both `last`, calculated as iterator, and `middle` as the stop point, when the output size is sufficient
 - stop points for both “subranges” (from middle to last and from first to middle) when the output size is less than the input one.

Possible implementation of envisioned serial algorithm:

```
struct rotate_copy_fn
{
    template<std::forward_iterator I, std::sentinel_for<I> S, std::forward_iterator O, std::sentinel_for<O> OutS>
    requires std::indirectly_copyable<I, O>
    constexpr std::ranges::rotate_copy_truncated_result<I, O>
        operator()(I first, I middle, S last, O result, OutS result_last) const
    {
        while (middle != last && result != result_last)
        {
            *result = *middle;
            ++middle;
            ++result;
        }

        while (first != middle && result != result_last)
        {
            *result = *first;
            ++first;
            ++result;
        }
        return {std::move(middle), std::move(first), std::move(result)};
    }

    template<std::ranges::forward_range R, std::ranges::forward_range OutR>
    requires std::indirectly_copyable<std::ranges::iterator_t<R>, std::ranges::iterator_t<OutR>>
    constexpr std::ranges::rotate_copy_truncated_result<std::ranges::borrowed_iterator_t<R>,
        std::ranges::borrowed_iterator_t<OutR>>
        operator()(R&& r, std::ranges::iterator_t<R> middle, OutR&& result_r) const
    {
        return (*this)(std::ranges::begin(r), std::move(middle),
```

```
        std::ranges::end(r), std::ranges::begin(result_r), std::ranges::end(result_r));
    }
};

inline constexpr rotate_copy_fn rotate_copy {};
```

Please note that we propose:

- in1 as a stop point in [middle, last].
- in2 as a stop point in [first, middle].

Even if both belong to the same input range those are just stop points in two “subranges”, so we don’t want users to think of them as of a valid `std::ranges::subrange`.

Caveat: Since C++26 we can imagine a code that compiles for both serial range algorithms and the parallel ones even with the proposed design, because of structured binding with a pack ([P1061R10]).

Let’s compare:

Serial rotate_copy vs parallel rotate_copy and structured binding with a pack

serial <code>ranges::rotate_copy</code>	Parallel <code>ranges::rotate_copy</code> with the proposed design in this paper
<pre>template <typename R, typename OutR> void my_algorithm(R&& in, OutR&& out) { auto middle = it; std::advance(middle, 3); // possibly to ignore rest... auto [in, ...rest] = std::ranges::rotate_copy(in, middle, std::ranges::begin(out)); // after a call sizeof...(rest) == 1 is // true }</pre>	<pre>template <typename R, typename OutR> void my_algorithm(R&& in, OutR&& out) { auto middle = it; std::advance(middle, 3); // possibly to ignore rest... auto [in, ...rest] = std::ranges::rotate_copy(std::execution::par, in, middle, out); // after a call sizeof...(rest) == 2 is true (with // this paper) }</pre>

We can imagine this code if people want to only store the first data member of the returned `struct` object and ignore the rest. With the code comparison above it’s even more important that `in1` is a stop point from `middle` to `last` because, as we stated above, this code compiles and works at runtime without surprising a user.

§ 3 reverse_copy recommended design

For `reverse_copy` the proposed algorithm in [P3179R7] always returns past the last element (in reversed order) for input. There is no “dual meaning” for any of returned iterators for the input. It works perfectly fine in isolation, however two concerns come in mind:

- different behavior at runtime compared to serial range counterpart, when output size is sufficient (serial one always returns last).
- for the envisioned serial `ranges::reverse_copy` with “range-as-the-output” we calculate `last` as the iterator but don’t return that. See [Introduction](#) for more information.

Consider the following example:

Serial vs Parallel reverse_copy

iterator-as-the-output reverse_copy	range-as-the-output reverse_copy
<pre>std::list v{1,2,3,4,5,6}; std::vector<int> out_v(6); // view is not common_range auto view = std::views::counted(v.begin(), 6); static_assert(!std::ranges::common_range<decltype(view)>); // iterator as the output auto res = std::ranges::reverse_copy(view, out_v.begin()); // res.in is last</pre>	<pre>std::list v{1,2,3,4,5,6}; std::vector<int> out_v(6); // view is not common_range auto view = std::views::counted(v.begin(), 6); static_assert(!std::ranges::common_range<decltype(view)>); // range as the output auto res = std::ranges::reverse_copy(view, it, out_v); // res.in is middle</pre>

iterator-as-the-output reverse_copy	range-as-the-output reverse_copy
<pre>auto val = std::reduce(view.begin(), res.in); // val is 21</pre>	<pre>auto val = std::reduce(view.begin(), res.in); // val is 0</pre>

Eventually, the story is similar with `rotate_copy`. All the following ruminations also apply for parallel `reverse_copy`:

- new return type vs `in_in_out_result`
- returning two different points of input range rather than a valid `std::ranges::subrange`
- structured binding with a parameter pack

The solution is also similar. We propose to return “alias-ed” `ranges::in_in_out_result`:

- `in1` is always last.
- `in2` represents a stop point.

The signatures are below:

```
template <class In, class Out>
using reverse_copy_truncated_result = std::ranges::in_in_out_result<In, In, Out>;

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        random_access_iterator O, sized_sentinel_for<O> OutS>
    requires indirectly_copyable<I, O>
    ranges::reverse_copy_truncated_result<I, O>
        ranges::reverse_copy(Ep&& exec, I first, S last, O result, OutS result_last);

template<execution-policy Ep, sized-random-access-range R, sized-random-access-range OutR>
    requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>>
    ranges::reverse_copy_truncated_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
        ranges::reverse_copy(Ep&& exec, R&& r, OutR&& result_r);
```

Possible implementation of envisioned serial algorithm:

```
struct reverse_copy_fn
{
    template<std::bidirectional_iterator I, std::sentinel_for<I> S, std::forward_iterator O, std::sentinel_for<O>
        OutS>
        requires std::indirectly_copyable<I, O>
        constexpr std::ranges::reverse_copy_truncated_result<I, O>
            operator()(I first, S last, O result, OutS result_last) const
        {
            auto res = std::ranges::next(first, last);
            auto last_iter = res;

            while (last_iter != first && result != result_last)
            {
                *result = *--last_iter;
                ++result;
            }

            return {last_iter, res, result};
        }

    template<std::ranges::bidirectional_range R, std::ranges::forward_range OutR>
        requires std::indirectly_copyable<std::ranges::iterator_t<R>, std::ranges::iterator_t<OutR>>
        constexpr std::ranges::reverse_copy_truncated_result<std::ranges::borrowed_iterator_t<R>,
            std::ranges::borrowed_iterator_t<OutR>>
            operator()(R&& r, OutR&& result_r) const
        {
            return (*this)(std::ranges::begin(r), std::ranges::end(r),
                std::ranges::begin(result_r), std::ranges::end(result_r));
        }
};

inline constexpr reverse_copy_fn reverse_copy {};
```

§ 4 Alternative design

There are the following alternatives to the proposed design:

- Keep [\[P3179R7\]](#) status quo. The concerns are:
 - The different behavior at runtime, when switching the code to parallel algorithms.

- No calculated `last` for future serial range algorithms with “range-as-the-output”.
- Keep `reverse_copy` as proposed in [P3179R7], return `last` for parallel `rotate_copy` when the output is sufficient. The concerns are:
 - `reverse_copy` remains the same, the concerns from the previous bullets apply.
 - `rotate_copy` becomes inconsistent with other parallel range algorithms with “range-as-the-output” because it “jumps” to last element instead of return past the last one.
 - The parallel `rotate_copy` calculates `last` but does not return it, so it likely means a different return type for serial “rangified” `rotate_copy` in the future.

If this proposal does not have a consensus, authors’ preference is to remove parallel `reverse_copy` and `rotate_copy` from [P3179R7] and add them later.

§ 5 Acknowledgements

- Thanks to Jonathan Mueller for having a fruitful discussion on `reverse_copy` and `rotate_copy` return type.

§ 6 References

[P1061R10] Barry Revzin, Jonathan Wakely. 2024-11-24. Structured Bindings can introduce a Pack.

<https://wg21.link/p1061r10>

[P3179R7] Ruslan Arutyunyan, Alexey Kukanov, Bryce Adelstein Lelbach. 2025-02-28. C++ parallel range algorithms.

<https://wg21.link/p3179r7>